In [1]:

```
%%writefile mantisv2_kit.py
# MantisV2 kit (same file as kit/mantisv2_kit.py in the group GitHub rep
o)
import argparse
import json
import os
import subprocess
import time
import traceback
import zipfile

import numpy as np
import pandas as pd
import requests
import torch
import torch.nn as nn
import torch.nn.functional as F
from sklearn.ensemble import RandomForestClassifier
from sklearn.linear_model import LogisticRegression
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder, StandardScaler

ALL_METHODS = ['rf', 'logreg', 'head', 'mixer', 'pool', 'pool_mixer', 'lo
ra']
SEEDS = (0, 1, 2)
DEVICE = 'cuda' if torch.cuda.is_available() else 'cpu'

TFC_ARTICLES = {'SleepEEG': 19930178, 'Epilepsy': 19930199, 'FD-A': 19930
205, 'FD-B': 19930226,
                'HAR': 19930244, 'Gesture': 19930247, 'EMG': 19930250, 'E
CG': 19930253}


def log(*msg):
    print(time.strftime('%H:%M:%S'), *msg, flush=True)


def _download(url, path):
    r = requests.get(url, timeout=900)
    r.raise_for_status()
```

```
    with open(path, 'wb') as fh:
        fh.write(r.content)


def _parse_ts(path):
    X, y, in_data = [], [], False
    with open(path, encoding='utf-8', errors='ignore') as fh:
        for line in fh:
            line = line.strip()
            if not line or line.startswith('#'):
                continue
            if line.lower().startswith('@data'):
                in_data = True
                continue
            if in_data:
                *channels, label = line.split(':')
                X.append([np.array([float(v) if v not in ('?', 'NaN') els
e np.nan for v in ch.split(',')],
                                   dtype=np.float32) for ch in channels])
                y.append(label)
    L = max(len(ch) for row in X for ch in row)
    out = np.zeros((len(X), len(X[0]), L), dtype=np.float32)
    for i, row in enumerate(X):
        for c, ch in enumerate(row):
            if len(ch) == L:
                out[i, c] = ch
            else:
                ch = np.nan_to_num(ch)
                out[i, c] = np.interp(np.linspace(0, len(ch) - 1, L), np.
arange(len(ch)), ch)
    return out, np.asarray(y)


def load_uea(name, data_dir):
    folder = os.path.join(data_dir, name)
    if not os.path.exists(folder):
        zip_path = os.path.join(data_dir, f'{name}.zip')
        log(f'downloading {name} from timeseriesclassification.com')
        _download(f'https://www.timeseriesclassification.com/aeon-toolki
t/{name}.zip', zip_path)
        with zipfile.ZipFile(zip_path) as z:
```

```
            z.extractall(folder)
        os.remove(zip_path)
    return load_ts_folder(name, folder)


def load_ts_folder(name, folder):
    out = {}
    for split in ['TRAIN', 'TEST']:
        found = [os.path.join(r, f) for r, _, fs in os.walk(folder) for f
in fs if f == f'{name}_{split}.ts']
        if not found:
            raise FileNotFoundError(f'{name}_{split}.ts not found in {fol
der} (the name must match exactly)')
        out[split.lower()] = _parse_ts(found[0])
    return out


def load_tfc(name, data_dir):
    folder = os.path.join(data_dir, f'TFC-{name}')
    os.makedirs(folder, exist_ok=True)
    files = None
    out = {}
    for split in ['train', 'test']:
        path = os.path.join(folder, f'{split}.pt')
        if not os.path.exists(path):
            if files is None:
                files = requests.get(f'https://api.figshare.com/v2/articl
es/{TFC_ARTICLES[name]}', timeout=30).json()['files']
            log(f'downloading TF-C {name} {split}.pt from figshare')
            _download([f['download_url'] for f in files if f['name'] ==
f'{split}.pt'][0], path)
        d = torch.load(path, weights_only=False)
        X, y = np.asarray(d['samples'], dtype=np.float32), np.asarray(d
['labels'])
        out[split] = (X, y)
    return out


def load_numpy(path):
    f = lambda n: os.path.join(path, n)
    if os.path.exists(f('X_train.npy')):
```

```
        return {'train': (np.load(f('X_train.npy')), np.load(f('y_train.n
py'), allow_pickle=True)),
                'test': (np.load(f('X_test.npy')), np.load(f('y_test.np
y'), allow_pickle=True))}
    X, y = np.load(f('X.npy')), np.load(f('y.npy'), allow_pickle=True)
    tr, te = train_test_split(np.arange(len(y)), test_size=0.25, stratify
=y, random_state=0)
    return {'train': (X[tr], y[tr]), 'test': (X[te], y[te])}


def load_pt(path):
    out = {}
    for split in ['train', 'test']:
        d = torch.load(os.path.join(path, f'{split}.pt'), weights_only=Fa
lse)
        out[split] = (np.asarray(d['samples'], dtype=np.float32), np.asar
ray(d['labels']))
    return out


def load_woods(path, split='ID', train_domains=None, test_domains=None, c
hannels_last=None):
    import h5py
    with h5py.File(path, 'r') as h:
        domains = sorted(h.keys())
        log(f'{os.path.basename(path)} domains: {domains}')
        if split == 'OOD':
            missing = [m for m in (train_domains or []) + (test_domains o
r []) if m not in domains]
            if missing or not train_domains or not test_domains:
                raise ValueError(f'for OOD give train_domains and test_do
mains from this list: {domains} '
                                 f'(not found: {missing})')
        Xs, ys, dom = [], [], []
        for k, m in enumerate(domains):
            if split == 'OOD' and m not in train_domains + test_domains:
                continue
            data, parts = h[f'{m}/data'], []
            last = channels_last if channels_last is not None else data.s
hape[1] > data.shape[2]
            for s in range(0, data.shape[0], 2000):
```

```
                x = data[s:s + 2000]
                parts.append(resize(np.transpose(x, (0, 2, 1)) if last el
se x))
            labels = h[f'{m}/labels'][:]
            labels = labels.argmax(1) if labels.ndim == 2 and labels.shap
e[1] > 1 else labels.ravel()
            Xs.append(np.concatenate(parts))
            ys.append(labels)
            dom += [m] * len(labels)
            log(f'  {m}: {Xs[-1].shape[0]} samples, {Xs[-1].shape[1]} cha
nnels')
    X, y, dom = np.concatenate(Xs), np.concatenate(ys), np.asarray(dom)
    if split == 'ID':
        N = len(y)
        perm = np.random.RandomState(0).permutation(N)
        tr, te = perm[:int(0.6375 * N)], perm[N - int(round(0.25 * N)):]
    else:
        tr, te = np.where(np.isin(dom, train_domains))[0], np.where(np.is
in(dom, test_domains))[0]
    return {'train': (X[tr], y[tr]), 'test': (X[te], y[te])}


SOURCES = ['uea', 'tfc', 'ts', 'numpy', 'pt', 'woods']


def load_dataset(name, source, path=None, data_dir='data', **kw):
    os.makedirs(data_dir, exist_ok=True)
    if source == 'uea':
        d = load_uea(name, data_dir)
    elif source == 'tfc':
        d = load_tfc(name, data_dir)
    elif source == 'ts':
        d = load_ts_folder(name, path or os.path.join(data_dir, name))
    elif source == 'numpy':
        d = load_numpy(path or os.path.join(data_dir, name))
    elif source == 'pt':
        d = load_pt(path or os.path.join(data_dir, name))
    elif source == 'woods':
        d = load_woods(path, **kw)
    else:
        raise ValueError(f'source must be one of {SOURCES}')
```

```
    for k, (X, y) in d.items():
        X = np.asarray(X)
        if X.dtype != np.float16:
            X = X.astype(np.float32)
        if X.ndim == 2:
            X = X[:, None, :]
        d[k] = (X, np.asarray(y).ravel())
    return d


_NET = None


def encoder():
    global _NET
    if _NET is None:
        from mantis.architecture import MantisV2
        net = MantisV2(device=DEVICE, return_transf_layer=2, output_token
='combined').from_pretrained('paris-noah/MantisV2')
        net.return_transf_layer, net.output_token = 2, 'combined'
        _NET = net.to(DEVICE).eval()
    return _NET


def resize(X, length=512):
    out = []
    for i in range(0, len(X), 2000):
        x = torch.tensor(np.nan_to_num(np.asarray(X[i:i + 2000], dtype=n
p.float32)))
        out.append(F.interpolate(x, size=length, mode='linear', align_cor
ners=False).numpy().astype(np.float16))
    return np.concatenate(out)


def features(Xr):
    from mantis.trainer import MantisTrainer
    model = MantisTrainer(device=DEVICE, network=encoder())
    N, C, _ = Xr.shape
    out = [model.transform(Xr[i:i + 2000].astype(np.float32)) for i in ra
nge(0, N, 2000)]
    return np.concatenate(out).reshape(N, C, 512).astype(np.float16)
```

```
@torch.no_grad()
def tokens(Xr, bs=1024):
    net = encoder()
    N, C, _ = Xr.shape
    flat = Xr.reshape(N * C, 1, 512)
    out = []
    for i in range(0, N * C, bs):
        h = net.tokgen_unit(torch.from_numpy(flat[i:i + bs]).to(DEVICE).f
loat())
        h = torch.cat([h, net.transf_unit.cls_token.expand(h.shape[0], 1,
-1)], 1)[:, None]
        layers = []
        for layer in net.transf_unit.transformer.layers:
            h = layer(h)
            t = h[:, 0]
            layers.append(torch.cat([t[:, 32:], t[:, :32].reshape(-1, 4,
8, 256).mean(2)], 1))
        out.append(torch.stack(layers, 1).half().cpu().numpy())
    return np.concatenate(out).reshape(N, C, 6, 5, 256)


class MixerV2(nn.Module):
    def __init__(self, C, K, mix, d=64, drop=0.3, chan_drop=0.1, d_in=51
2):
        super().__init__()
        self.mix, self.chan_drop = mix, chan_drop
        self.proj = nn.Sequential(nn.LayerNorm(d_in), nn.Linear(d_in, d))
        if mix:
            self.chan_emb = nn.Parameter(torch.zeros(1, C, d))
            self.norm = nn.LayerNorm(d)
            self.attn = nn.MultiheadAttention(d, 2, dropout=drop, batch_f
irst=True)
            self.gate = nn.Parameter(torch.zeros(1))
        self.out = nn.Sequential(nn.Dropout(drop), nn.Linear(C * d, K))

    def forward(self, Z):
        z = self.proj(Z.float())
        if self.training and self.chan_drop > 0:
            z = z * (torch.rand(z.shape[0], z.shape[1], 1, device=z.devic
```

```
e) > self.chan_drop).float()
        if self.mix:
            h = self.norm(z + self.chan_emb)
            z = z + torch.tanh(self.gate) * self.attn(h, h, h, need_weigh
ts=False)[0]
        return self.out(z.flatten(1))


class PoolHead(nn.Module):
    def __init__(self, C, K, mix, L=6, heads=4):
        super().__init__()
        self.layer_w = nn.Parameter(torch.zeros(L))
        self.norm = nn.LayerNorm(256)
        self.query = nn.Parameter(torch.randn(heads, 256) * 0.02)
        self.head = MixerV2(C, K, mix, d_in=heads * 256)

    def forward(self, T):
        w = torch.softmax(self.layer_w, 0)
        t = (self.norm(T.float()) * w[:, None, None]).sum(2)
        a = torch.softmax(t @ self.query.T / 16, 2)
        return self.head((a.transpose(2, 3) @ t).flatten(2))


class LoRA(nn.Module):
    def __init__(self, base, r=8, alpha=16):
        super().__init__()
        self.base, self.scale = base, alpha / r
        self.A = nn.Parameter(torch.randn(r, base.in_features, device=bas
e.weight.device) * 0.01)
        self.B = nn.Parameter(torch.zeros(base.out_features, r, device=ba
se.weight.device))

    def forward(self, x):
        return self.base(x) + (x @ self.A.T @ self.B.T) * self.scale


class LoraModel(nn.Module):
    def __init__(self, C, K, r=8):
        super().__init__()
        from mantis.architecture import MantisV2
        self.enc = MantisV2(device=DEVICE, return_transf_layer=2, output_
```

```
token='combined')
        self.enc.load_state_dict(encoder().state_dict())
        self.enc.remove_transf_layers()
        for p in self.enc.parameters():
            p.requires_grad = False
        for layer in self.enc.transf_unit.transformer.layers:
            layer.attention.wQKV = LoRA(layer.attention.wQKV, r)
            layer.attention.wO = LoRA(layer.attention.wO, r)
        self.head = MixerV2(C, K, True)

    def forward(self, X):
        B, C, L = X.shape
        return self.head(self.enc(X.reshape(B * C, 1, L).float()).reshape
(B, C, 512))


def fit(model, A, y, tr_idx, te_idx, seed, epochs, patience, bs, lr=1e-3,
amp=False):
    amp = amp and DEVICE == 'cuda'
    fit_idx, va_idx = train_test_split(tr_idx, test_size=0.2, stratify=y
[tr_idx], random_state=seed)
    params = [p for p in model.parameters() if p.requires_grad]
    opt = torch.optim.AdamW(params, lr=lr, weight_decay=0.05)
    sched = torch.optim.lr_scheduler.CosineAnnealingLR(opt, epochs)
    scaler = torch.amp.GradScaler('cuda', enabled=amp)
    batch = lambda idx: torch.from_numpy(A[idx]).to(DEVICE)

    def predict(idx):
        model.eval()
        preds = []
        with torch.no_grad(), torch.autocast(DEVICE, dtype=torch.float16,
enabled=amp):
            for i in range(0, len(idx), 256):
                preds.append(model(batch(idx[i:i + 256])).argmax(1).cpu
().numpy())
        return np.concatenate(preds)

    best, best_state, wait = -1, None, 0
    for ep in range(epochs):
        model.train()
        perm = np.random.permutation(fit_idx)
```

```
        for i in range(0, len(perm), bs):
            b = perm[i:i + bs]
            with torch.autocast(DEVICE, dtype=torch.float16, enabled=am
p):
                loss = F.cross_entropy(model(batch(b)), torch.from_numpy
(y[b]).long().to(DEVICE), label_smoothing=0.1)
            opt.zero_grad()
            scaler.scale(loss).backward()
            scaler.step(opt)
            scaler.update()
        sched.step()
        va = np.mean(predict(va_idx) == y[va_idx])
        if va >= best:
            best, wait = va, 0
            best_state = {k: v.detach().clone() for k, v in model.state_d
ict().items()}
        else:
            wait += 1
            if wait >= patience:
                break
    model.load_state_dict(best_state)
    return float(np.mean(predict(te_idx) == y[te_idx])), ep + 1


def settings(n_train, quick=False):
    if quick:
        return dict(head=dict(epochs=3, patience=2, bs=64), pool=dict(epo
chs=2, patience=1, bs=64),
                    lora=dict(epochs=1, patience=1, bs=32))
    if n_train < 10000:
        return dict(head=dict(epochs=100, patience=20, bs=64), pool=dict
(epochs=100, patience=20, bs=64),
                    lora=dict(epochs=30, patience=8, bs=32))
    return dict(head=dict(epochs=40, patience=8, bs=256), pool=dict(epoch
s=20, patience=5, bs=256),
                lora=dict(epochs=4, patience=2, bs=64))


def run(dataset, source='uea', path=None, methods=None, seeds=SEEDS, out_
dir='results', data_dir='data',
        quick=False, paper=None, **load_kw):
```

```
    methods = methods or ALL_METHODS
    os.makedirs(out_dir, exist_ok=True)
    csv = os.path.join(out_dir, f'{dataset}_results.csv')
    t_all = time.time()
    log(f'{dataset}: device {DEVICE}' + (f' ({torch.cuda.get_device_name
(0)})' if DEVICE == 'cuda' else
                                          ' - no GPU, the trained methods
will be slow'))

    d = load_dataset(dataset, source, path, data_dir, **load_kw)
    (Xa, ya), (Xb, yb) = d['train'], d['test']
    le = LabelEncoder().fit(np.concatenate([ya, yb]))
    y = le.transform(np.concatenate([ya, yb]))
    tr, te = np.arange(len(ya)), np.arange(len(ya), len(ya) + len(yb))
    L0 = Xa.shape[2]
    Xr = resize(np.concatenate([Xa, Xb]))
    del Xa, Xb, d
    N, C, _ = Xr.shape
    K = len(le.classes_)
    S = settings(len(tr), quick)
    log(f'{dataset}: train {len(tr)}, test {len(te)}, channels {C}, origi
nal length {L0}, classes {K}')

    rows = []

    def record(method, accs, t0, **extra):
        row = dict(dataset=dataset, method=method, mean=float(np.mean(acc
s)), std=float(np.std(accs)),
                   per_seed=json.dumps([round(a, 4) for a in accs]), pape
r=paper, n_train=len(tr), n_test=len(te),
                   channels=C, classes=K, time_s=round(time.time() - t0),
**extra)
        rows.append(row)
        pd.DataFrame(rows).to_csv(csv, index=False)
        log(f'{dataset:16s} {method:12s} {row["mean"]:.4f} +/- {row["st
d"]:.4f}  {extra or ""}  {row["time_s"]}s')

    def safe(fn):
        try:
            fn()
        except Exception as e:
```

```
            traceback.print_exc()
            log('FAILED:', type(e).__name__, e)

    Z = None
    if {'rf', 'logreg', 'head', 'mixer'} & set(methods):
        t0 = time.time()
        Z = features(Xr)
        log(f'features {Z.shape} in {time.time() - t0:.0f}s')
        Zflat = Z.reshape(N, -1).astype(np.float32)

    if 'rf' in methods:
        def m():
            t0, accs = time.time(), []
            for s in seeds:
                rf = RandomForestClassifier(n_estimators=200, n_jobs=-1,
random_state=s).fit(Zflat[tr], y[tr])
                accs.append(float(np.mean(rf.predict(Zflat[te]) == y[t
e])))
            record('rf', accs, t0)
        safe(m)

    if 'logreg' in methods:
        def m():
            t0 = time.time()
            sc = StandardScaler().fit(Zflat[tr])
            clf = LogisticRegression(max_iter=500).fit(sc.transform(Zflat
[tr]), y[tr])
            record('logreg', [float(np.mean(clf.predict(sc.transform(Zfla
t[te])) == y[te]))], t0)
        safe(m)

    def heads(name, A, make, cfg, extra_fn, amp=False):
        def m():
            t0, accs, eps, extras = time.time(), [], [], []
            for s in seeds:
                torch.manual_seed(s)
                np.random.seed(s)
                model = make().to(DEVICE)
                acc, ep = fit(model, A, y, tr, te, s, amp=amp, **cfg)
                accs.append(acc)
                eps.append(ep)
```

```
                extras.append(extra_fn(model))
            record(name, accs, t0, epochs=json.dumps(eps), **{k: json.dum
ps([e[k] for e in extras]) for k in extras[0]})
        safe(m)

    gate = lambda h: round(torch.tanh(h.gate).item(), 3) if h.mix else 0.
0
    if 'head' in methods:
        heads('head', Z, lambda: MixerV2(C, K, False), S['head'], lambda
m: {})
    if 'mixer' in methods:
        heads('mixer', Z, lambda: MixerV2(C, K, True), S['head'], lambda
m: {'gate': gate(m)})
    del Z

    if {'pool', 'pool_mixer'} & set(methods):
        gb = N * C * 6 * 5 * 256 * 2 / 1e9
        if gb > 20:
            log(f'skipping pool methods: tokens would need {gb:.0f} GB of
RAM')
        else:
            t0 = time.time()
            T = tokens(Xr)
            log(f'tokens {T.shape} ({gb:.1f} GB) in {time.time() - t0:.0
f}s')
            lw = lambda m: {'gate': gate(m.head),
                            'layer_weights': [round(v, 3) for v in torch.
softmax(m.layer_w, 0).tolist()]}
            if 'pool' in methods:
                heads('pool', T, lambda: PoolHead(C, K, False), S['poo
l'], lw)
            if 'pool_mixer' in methods:
                heads('pool_mixer', T, lambda: PoolHead(C, K, True), S['p
ool'], lw)
            del T

    if 'lora' in methods:
        heads('lora', Xr, lambda: LoraModel(C, K), S['lora'], lambda m:
{'gate': gate(m.head)}, amp=True)

    df = pd.DataFrame(rows)
```

```
    log(f'{dataset} done in {(time.time() - t_all) / 60:.1f} min, saved {csv}')
    print(df[['dataset', 'method', 'mean', 'std', 'per_seed']].round(4).to_string(index=False))
    return df
```

```
Writing mantisv2_kit.py
```

In [2]:

```
# Runner for datasets 1-6 and 13-18. PART 1: SCP1, SCP2, EMOPain, MP8, MP
50, HHAR-ID/OOD; PART 2: PCL-ID/OOD; PART 3: SEDFx-ID; PART 4: SEDFx-OOD
# Ego4D (dataset 1) is not run: it needs a signed licence agreement.
PART = 1
!pip install -q mantis-tsfm gdown
import os, zipfile, subprocess, glob, numpy as np, pandas as pd, gdown
import mantisv2_kit as kit
DATA, OUT = '/tmp/data', '/kaggle/working/results'
os.makedirs(DATA, exist_ok=True)

def get(fn):
    try:
        fn()
    except Exception as e:
        print('DATA FAILED:', type(e).__name__, e, flush=True)

def uea(n):
    subprocess.run(['wget', '-q', '-O', f'{DATA}/{n}.zip', f'https://www.
timeseriesclassification.com/aeon-toolkit/{n}.zip'], check=True)
    zipfile.ZipFile(f'{DATA}/{n}.zip').extractall(f'{DATA}/{n}')

def mp():
    # MP8 / MP50 as released by Sungu Isiacik et al. (2025), github.com/m
lgig/FM4MTSC
    gdown.download_folder(id='10Hvep0hk9naj_nRk4JMiXHMeV1vw0lcd', output=
f'{DATA}/fm4mtsc', quiet=True)
    for n in ['MP8', 'MP50']:
        d = np.load(f'{DATA}/fm4mtsc/{n}.npy', allow_pickle=True).item()
        print(n, list(d), {k: np.shape(v['X']) for k, v in d.items()}, fl
ush=True)
        os.makedirs(f'{DATA}/{n}', exist_ok=True)
        for s in ['train', 'test']:
            np.save(f'{DATA}/{n}/X_{s}.npy', np.asarray(d[s]['X'], dtype=
np.float32))
            np.save(f'{DATA}/{n}/y_{s}.npy', np.asarray(d[s]['y']))

def woods(gid, name):
    # WOODS benchmark files (github.com/jc-audet/WOODS, woods/download.p
y)
    gdown.download(id=gid, output=f'{DATA}/{name}.zip', quiet=True)
```

```
    zipfile.ZipFile(f'{DATA}/{name}.zip').extractall(f'{DATA}/{name}')
    os.remove(f'{DATA}/{name}.zip')

def h5(name):
    return [os.path.join(r, f) for r, _, fs in os.walk(f'{DATA}/{name}')
for f in fs if f.endswith('.h5')][0]

if PART == 1:
    for n in ['SelfRegulationSCP1', 'SelfRegulationSCP2', 'EMOPain']:
        get(lambda: uea(n))
    get(mp)
    get(lambda: woods('1Z3IcrCE-o77p4YrvkCy-Y-0CgCyxVHet', 'HHAR'))
    JOBS = [dict(name='SelfRegulationSCP1', source='uea', paper=0.8134),
            dict(name='SelfRegulationSCP2', source='uea', paper=0.5167),
            dict(name='EMOPain', source='uea', paper=0.8798),
            dict(name='MP8', source='numpy', paper=0.6857),
            dict(name='MP50', source='numpy', paper=0.7345),
            dict(name='HHAR-ID', source='woods', h5='HHAR', split='ID', p
aper=0.9845),
            dict(name='HHAR-OOD', source='woods', h5='HHAR', split='OOD',
train_domains=['nexus4', 's3', 's3mini'],
                 test_domains=['lgwatch'], paper=0.5822)]
elif PART == 2:
    get(lambda: woods('118DNxHpzeJwVTM22wzZhSiOuDsno0nay', 'PCL'))
    JOBS = [dict(name='PCL-ID', source='woods', h5='PCL', split='ID', pap
er=0.5716),
            dict(name='PCL-OOD', source='woods', h5='PCL', split='OOD', t
rain_domains=['Cho2017'],
                 test_domains=['Lee2019_MI'], paper=0.5311)]
else:
    get(lambda: woods('15j_bsiOmMJb42mG712Vhv3jZ4MQSOgoT', 'SEDFx'))
    JOBS = [dict(name='SEDFx-ID', source='woods', h5='SEDFx', split='ID',
paper=0.8000)] if PART == 3 else \
           [dict(name='SEDFx-OOD', source='woods', h5='SEDFx', split='OO
D', train_domains=['Age 20-40', 'Age 40-60'],
                 test_domains=['Age 80-100'], paper=0.7636)]

print(sorted(os.listdir(DATA)), flush=True)
for j in JOBS:
    j = dict(j)
    try:
```

```
        if 'h5' in j:
            j['path'] = h5(j.pop('h5'))
        kit.run(j.pop('name'), j.pop('source'), j.pop('path', None), out_
dir=OUT, data_dir=DATA, **j)
    except Exception as e:
        print('RUN FAILED:', type(e).__name__, e, flush=True)
files = glob.glob(f'{OUT}/*_results.csv')
if files:
    df = pd.concat([pd.read_csv(f) for f in files])
    df.to_csv(f'{OUT}/part{PART}_all_results.csv', index=False)
    print(df.pivot_table(index='dataset', columns='method', values='mea
n').round(4).to_string())
```

```
MP8 ['train', 'test'] {'train': (1426, 8, 161), 'test': (595, 8, 161)}
MP50 ['train', 'test'] {'train': (1426, 50, 161), 'test': (595, 50, 16
1)}
['EMOPain', 'EMOPain.zip', 'HHAR', 'MP50', 'MP8', 'SelfRegulationSCP
1', 'SelfRegulationSCP1.zip', 'SelfRegulationSCP2', 'SelfRegulationSCP
2.zip', 'fm4mtsc']
22:50:12 SelfRegulationSCP1: device cuda (Tesla T4)
22:50:13 SelfRegulationSCP1: train 268, test 293, channels 6, original
length 896, classes 2

Warning: You are sending unauthenticated requests to the HF Hub. Pleas
e set a HF_TOKEN to enable higher rate limits and faster downloads.
```

```
22:50:24 features (561, 6, 512) in 10s
22:50:26 SelfRegulationSCP1 rf           0.8134 +/- 0.0016    2s
22:50:27 SelfRegulationSCP1 logreg       0.8669 +/- 0.0000    1s
22:50:36 SelfRegulationSCP1 head         0.8305 +/- 0.0237  {'epochs':
'[61, 40, 100]'}  9s
22:50:39 SelfRegulationSCP1 mixer        0.8316 +/- 0.0458  {'epochs':
'[64, 37, 68]', 'gate': '[0.036, -0.027, -0.051]'}  3s
22:50:40 tokens (561, 6, 6, 5, 256) (0.1 GB) in 1s
22:50:45 SelfRegulationSCP1 pool         0.7998 +/- 0.0259  {'epochs':
'[100, 32, 50]', 'gate': '[0.0, 0.0, 0.0]', 'layer_weights': '[[0.165,
0.167, 0.169, 0.167, 0.164, 0.167], [0.166, 0.166, 0.167, 0.167, 0.16
7, 0.167], [0.166, 0.167, 0.17, 0.166, 0.165, 0.167]]'}  5s
22:50:51 SelfRegulationSCP1 pool_mixer   0.7952 +/- 0.0145  {'epochs':
'[100, 48, 71]', 'gate': '[0.052, -0.016, -0.054]', 'layer_weights':
'[[0.165, 0.168, 0.169, 0.167, 0.164, 0.166], [0.166, 0.167, 0.168, 0.
165, 0.167, 0.168], [0.167, 0.168, 0.171, 0.166, 0.163, 0.165]]'}  6s
22:51:13 SelfRegulationSCP1 lora         0.8544 +/- 0.0153  {'epochs':
'[24, 14, 30]', 'gate': '[0.002, 0.0, -0.005]'}  22s
22:51:13 SelfRegulationSCP1 done in 1.0 min, saved /kaggle/working/res
ults/SelfRegulationSCP1_results.csv
           dataset     method   mean    std                 per_seed
SelfRegulationSCP1         rf 0.8134 0.0016 [0.8157, 0.8123, 0.8123]
SelfRegulationSCP1     logreg 0.8669 0.0000                 [0.8669]
SelfRegulationSCP1       head 0.8305 0.0237  [0.8294, 0.802, 0.8601]
SelfRegulationSCP1      mixer 0.8316 0.0458 [0.8532, 0.7679, 0.8737]
SelfRegulationSCP1       pool 0.7998 0.0259 [0.8089, 0.7645, 0.8259]
SelfRegulationSCP1 pool_mixer 0.7952 0.0145 [0.8055, 0.8055, 0.7747]
SelfRegulationSCP1       lora 0.8544 0.0153 [0.8328, 0.8669, 0.8635]
22:51:13 SelfRegulationSCP2: device cuda (Tesla T4)
22:51:13 SelfRegulationSCP2: train 200, test 180, channels 7, original
length 1152, classes 2
22:51:14 features (380, 7, 512) in 0s
22:51:16 SelfRegulationSCP2 rf           0.5148 +/- 0.0228    2s
22:51:17 SelfRegulationSCP2 logreg       0.5111 +/- 0.0000    2s
22:51:18 SelfRegulationSCP2 head         0.5019 +/- 0.0277  {'epochs':
'[22, 57, 56]'}  1s
22:51:20 SelfRegulationSCP2 mixer        0.4944 +/- 0.0091  {'epochs':
'[76, 30, 33]', 'gate': '[0.02, 0.001, 0.006]'}  2s
22:51:21 tokens (380, 7, 6, 5, 256) (0.0 GB) in 1s
22:51:22 SelfRegulationSCP2 pool         0.4963 +/- 0.0146  {'epochs':
```

'[28, 24, 37]', 'gate': '[0.0, 0.0, 0.0]', 'layer_weights': '[[0.167, 0.167, 0.166, 0.166, 0.167, 0.167], [0.168, 0.166, 0.166, 0.167, 0.167, 0.167], [0.167, 0.166, 0.166, 0.167, 0.167, 0.167]]'} 1s
22:51:24 SelfRegulationSCP2 pool_mixer   0.5352 +/- 0.0139 {'epochs': '[43, 28, 29]', 'gate': '[0.0, 0.004, 0.006]', 'layer_weights': '[[0.167, 0.166, 0.166, 0.166, 0.167, 0.167], [0.167, 0.167, 0.166, 0.166, 0.167, 0.167], [0.168, 0.167, 0.166, 0.166, 0.167, 0.167]]'} 2s
22:51:37 SelfRegulationSCP2 lora   0.5148 +/- 0.0547 {'epochs': '[25, 9, 13]', 'gate': '[0.0, -0.002, 0.001]'} 13s
22:51:37 SelfRegulationSCP2 done in 0.4 min, saved /kaggle/working/results/SelfRegulationSCP2_results.csv

dataset	method	mean	std	per_seed
SelfRegulationSCP2	rf	0.5148	0.0228	[0.5111, 0.4889, 0.5444]
SelfRegulationSCP2	logreg	0.5111	0.0000	[0.5111]
SelfRegulationSCP2	head	0.5019	0.0277	[0.4722, 0.4944, 0.5389]
SelfRegulationSCP2	mixer	0.4944	0.0091	[0.4833, 0.5056, 0.4944]
SelfRegulationSCP2	pool	0.4963	0.0146	[0.4889, 0.4833, 0.5167]
SelfRegulationSCP2	pool_mixer	0.5352	0.0139	[0.5389, 0.55, 0.5167]
SelfRegulationSCP2	lora	0.5148	0.0547	[0.5222, 0.4444, 0.5778]

22:51:37 EMOPain: device cuda (Tesla T4)
22:51:40 EMOPain: train 968, test 355, channels 30, original length 200, classes 3
22:51:46 features (1323, 30, 512) in 6s
22:52:07 EMOPain   rf   0.8826 +/- 0.0058   21s
22:52:24 EMOPain   logreg   0.8310 +/- 0.0000   17s
22:52:33 EMOPain   head   0.8469 +/- 0.0163 {'epochs': '[47, 81, 97]'} 8s
22:52:43 EMOPain   mixer   0.8535 +/- 0.0120 {'epochs': '[47, 54, 90]', 'gate': '[0.077, -0.072, -0.112]'} 11s
22:52:52 tokens (1323, 30, 6, 5, 256) (0.6 GB) in 9s
22:53:25 EMOPain   pool   0.8423 +/- 0.0023 {'epochs': '[47, 57, 45]', 'gate': '[0.0, 0.0, 0.0]', 'layer_weights': '[[0.16, 0.159, 0.158, 0.159, 0.182, 0.183], [0.155, 0.154, 0.155, 0.157, 0.191, 0.188], [0.157, 0.157, 0.159, 0.157, 0.186, 0.186]]'} 33s
22:53:53 EMOPain   pool_mixer   0.8394 +/- 0.0105 {'epochs': '[33, 52, 31]', 'gate': '[-0.012, 0.049, 0.0]', 'layer_weights': '[[0.161, 0.161, 0.16, 0.162, 0.178, 0.179], [0.155, 0.158, 0.157, 0.157, 0.187, 0.187], [0.16, 0.16, 0.158, 0.158, 0.18, 0.183]]'} 28s
22:55:56 FAILED: OutOfMemoryError CUDA out of memory. Tried to allocate 3.75 GiB. GPU 0 has a total capacity of 14.56 GiB of which 660.81 MiB is free. Including non-PyTorch memory, this process has 13.91 GiB me

```
mory in use. Of the allocated memory 11.40 GiB is allocated by PyTorc
h, and 2.37 GiB is reserved by PyTorch but unallocated. If reserved bu
t unallocated memory is large try setting PYTORCH_CUDA_ALLOC_CONF=expa
ndable_segments:True to avoid fragmentation.  See documentation for Me
mory Management  (https://docs.pytorch.org/docs/stable/notes/cuda.html
#optimizing-memory-usage-with-pytorch-cuda-alloc-conf)
22:55:56 EMOPain done in 4.3 min, saved /kaggle/working/results/EMOPai
n_results.csv
dataset     method   mean    std                 per_seed
EMOPain         rf 0.8826 0.0058 [0.8901, 0.8817, 0.8761]
EMOPain     logreg 0.8310 0.0000                  [0.831]
EMOPain       head 0.8469 0.0163 [0.8254, 0.8507, 0.8648]
EMOPain      mixer 0.8535 0.0120   [0.8366, 0.862, 0.862]
EMOPain       pool 0.8423 0.0023 [0.8423, 0.8394, 0.8451]
EMOPain pool_mixer 0.8394 0.0105 [0.8254, 0.8423, 0.8507]
22:55:56 MP8: device cuda (Tesla T4)
22:55:56 MP8: train 1426, test 595, channels 8, original length 161, c
lasses 4
```

```
Traceback (most recent call last):
  File "/kaggle/working/mantisv2_kit.py", line 390, in safe
    fn()
    ~~^^
  File "/kaggle/working/mantisv2_kit.py", line 426, in m
    acc, ep = fit(model, A, y, tr, te, s, amp=amp, **cfg)
              ~~~^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/kaggle/working/mantisv2_kit.py", line 342, in fit
    return float(np.mean(predict(te_idx) == y[te_idx])), ep + 1
                         ~~~~~~~^^^^^^^^
  File "/kaggle/working/mantisv2_kit.py", line 317, in predict
    preds.append(model(batch(idx[i:i + 256])).argmax(1).cpu().numpy())
                 ~~~~~^^^^^^^^^^^^^^^^^^^^^^
  File "/usr/local/lib/python3.13/dist-packages/torch/nn/modules/modul
e.py", line 1779, in _wrapped_call_impl
    return self._call_impl(*args, **kwargs)
           ~~~~~~~~~~~~~~~~^^^^^^^^^^^^^^^^^^
  File "/usr/local/lib/python3.13/dist-packages/torch/nn/modules/modul
e.py", line 1790, in _call_impl
    return forward_call(*args, **kwargs)
  File "/kaggle/working/mantisv2_kit.py", line 300, in forward
    return self.head(self.enc(X.reshape(B * C, 1, L).float()).reshape
(B, C, 512))
                     ~~~~~~~~^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/usr/local/lib/python3.13/dist-packages/torch/nn/modules/modul
e.py", line 1779, in _wrapped_call_impl
    return self._call_impl(*args, **kwargs)
           ~~~~~~~~~~~~~~~~^^^^^^^^^^^^^^^^^^
  File "/usr/local/lib/python3.13/dist-packages/torch/nn/modules/modul
e.py", line 1790, in _call_impl
    return forward_call(*args, **kwargs)
  File "/usr/local/lib/python3.13/dist-packages/mantis/architecture/ve
rsion2.py", line 251, in forward
    x_embeddings = self.tokgen_unit(x)
  File "/usr/local/lib/python3.13/dist-packages/torch/nn/modules/modul
e.py", line 1779, in _wrapped_call_impl
    return self._call_impl(*args, **kwargs)
           ~~~~~~~~~~~~~~~~^^^^^^^^^^^^^^^^^^
  File "/usr/local/lib/python3.13/dist-packages/torch/nn/modules/modul
e.py", line 1790, in _call_impl
```

```
    return forward_call(*args, **kwargs)
  File "/usr/local/lib/python3.13/dist-packages/mantis/architecture/version2.py", line 88, in forward
    embedding = self.layer_norms[i](embedding)
  File "/usr/local/lib/python3.13/dist-packages/torch/nn/modules/module.py", line 1779, in _wrapped_call_impl
    return self._call_impl(*args, **kwargs)
           ~~~~~~~~~~~~~~~~~~^^^^^^^^^^^^^^^^^^
  File "/usr/local/lib/python3.13/dist-packages/torch/nn/modules/module.py", line 1790, in _call_impl
    return forward_call(*args, **kwargs)
  File "/usr/local/lib/python3.13/dist-packages/torch/nn/modules/normalization.py", line 229, in forward
    return F.layer_norm(
           ~~~~~~~~~~~~^
        input, self.normalized_shape, self.weight, self.bias, self.eps
        ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
    )
    ^
  File "/usr/local/lib/python3.13/dist-packages/torch/nn/functional.py", line 2935, in layer_norm
    return torch.layer_norm(
           ~~~~~~~~~~~~~~~~^
        input, normalized_shape, weight, bias, eps, torch.backends.cudnn.enabled
        ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
    )
    ^
torch.OutOfMemoryError: CUDA out of memory. Tried to allocate 3.75 GiB. GPU 0 has a total capacity of 14.56 GiB of which 660.81 MiB is free. Including non-PyTorch memory, this process has 13.91 GiB memory in use. Of the allocated memory 11.40 GiB is allocated by PyTorch, and 2.37 GiB is reserved by PyTorch but unallocated. If reserved but unallocated memory is large try setting PYTORCH_CUDA_ALLOC_CONF=expandable_segments:True to avoid fragmentation.  See documentation for Memory Management  (https://docs.pytorch.org/docs/stable/notes/cuda.html#optimizing-memory-usage-with-pytorch-cuda-alloc-conf)
```

```
22:55:59 features (2021, 8, 512) in 3s
22:56:16 MP8              rf           0.6756 +/- 0.0063    17s
22:56:21 MP8              logreg       0.6521 +/- 0.0000    5s
22:56:32 MP8              head         0.6852 +/- 0.0165  {'epochs':
'[100, 98, 95]'}  12s
22:56:51 MP8              mixer        0.7109 +/- 0.0117  {'epochs':
'[92, 81, 94]', 'gate': '[-0.125, -0.14, -0.128]'}  18s
22:56:54 tokens (2021, 8, 6, 5, 256) (0.2 GB) in 4s
22:57:21 MP8              pool         0.6359 +/- 0.0130  {'epochs':
'[79, 90, 69]', 'gate': '[0.0, 0.0, 0.0]', 'layer_weights': '[[0.141,
0.143, 0.143, 0.155, 0.2, 0.217], [0.146, 0.135, 0.149, 0.149, 0.207,
0.215], [0.145, 0.138, 0.146, 0.158, 0.2, 0.212]]'}  27s
22:57:51 MP8              pool_mixer   0.6291 +/- 0.0042  {'epochs':
'[53, 69, 89]', 'gate': '[0.201, -0.195, 0.215]', 'layer_weights':
'[[0.148, 0.139, 0.15, 0.16, 0.203, 0.201], [0.146, 0.147, 0.15, 0.15
9, 0.197, 0.201], [0.153, 0.139, 0.146, 0.159, 0.198, 0.205]]'}  30s
23:00:56 MP8              lora         0.6947 +/- 0.0140  {'epochs':
'[30, 28, 25]', 'gate': '[0.105, 0.116, -0.09]'}  185s
23:00:56 MP8 done in 5.0 min, saved /kaggle/working/results/MP8_result
s.csv
dataset     method   mean    std                 per_seed
    MP8         rf 0.6756 0.0063  [0.6689, 0.684, 0.6739]
    MP8     logreg 0.6521 0.0000                 [0.6521]
    MP8       head 0.6852 0.0165 [0.6874, 0.6639, 0.7042]
    MP8      mixer 0.7109 0.0117 [0.7126, 0.7244, 0.6958]
    MP8       pool 0.6359 0.0130 [0.6235, 0.6538, 0.6303]
    MP8 pool_mixer 0.6291 0.0042 [0.6235, 0.6336, 0.6303]
    MP8       lora 0.6947 0.0140 [0.6874, 0.7143, 0.6824]
23:00:56 MP50: device cuda (Tesla T4)
23:00:56 MP50: train 1426, test 595, channels 50, original length 161,
classes 4
23:01:13 features (2021, 50, 512) in 16s
23:01:49 MP50             rf           0.7092 +/- 0.0049    37s
23:01:53 MP50             logreg       0.6689 +/- 0.0000    4s
23:02:09 MP50             head         0.7008 +/- 0.0027  {'epochs':
'[84, 89, 79]'}  15s
23:02:32 MP50             mixer        0.7126 +/- 0.0119  {'epochs':
'[57, 100, 100]', 'gate': '[-0.067, 0.116, -0.1]'}  23s
23:02:56 tokens (2021, 50, 6, 5, 256) (1.6 GB) in 24s
23:04:56 MP50             pool         0.7048 +/- 0.0107  {'epochs':
```

'[87, 76, 53]', 'gate': '[0.0, 0.0, 0.0]', 'layer_weights': '[[0.159, 0.167, 0.164, 0.166, 0.172, 0.172], [0.156, 0.164, 0.16, 0.165, 0.178, 0.178], [0.16, 0.164, 0.161, 0.166, 0.174, 0.175]]'} 121s
23:07:40 MP50 pool_mixer 0.6930 +/- 0.0057 {'epochs': '[100, 99, 80]', 'gate': '[0.173, 0.166, 0.14]', 'layer_weights': '[[0.16, 0.166, 0.164, 0.168, 0.171, 0.171], [0.159, 0.164, 0.16, 0.166, 0.175, 0.175], [0.162, 0.166, 0.161, 0.167, 0.173, 0.171]]'} 163s
23:07:51 FAILED: OutOfMemoryError CUDA out of memory. Tried to allocate 6.25 GiB. GPU 0 has a total capacity of 14.56 GiB of which 3.50 GiB is free. Including non-PyTorch memory, this process has 11.06 GiB memory in use. Of the allocated memory 6.46 GiB is allocated by PyTorch, and 4.46 GiB is reserved by PyTorch but unallocated. If reserved but unallocated memory is large try setting PYTORCH_CUDA_ALLOC_CONF=expandable_segments:True to avoid fragmentation. See documentation for Memory Management (https://docs.pytorch.org/docs/stable/notes/cuda.html#optimizing-memory-usage-with-pytorch-cuda-alloc-conf)
23:07:51 MP50 done in 6.9 min, saved /kaggle/working/results/MP50_results.csv

dataset	method	mean	std	per_seed
MP50	rf	0.7092	0.0049	[0.7042, 0.7076, 0.716]
MP50	logreg	0.6689	0.0000	[0.6689]
MP50	head	0.7008	0.0027	[0.7042, 0.7008, 0.6975]
MP50	mixer	0.7126	0.0119	[0.6958, 0.721, 0.721]
MP50	pool	0.7048	0.0107	[0.7193, 0.7008, 0.6941]
MP50	pool_mixer	0.6930	0.0057	[0.7008, 0.6908, 0.6874]

23:07:51 HHAR-ID: device cuda (Tesla T4)

```
Traceback (most recent call last):
  File "/kaggle/working/mantisv2_kit.py", line 390, in safe
    fn()
    ~~^^
  File "/kaggle/working/mantisv2_kit.py", line 426, in m
    acc, ep = fit(model, A, y, tr, te, s, amp=amp, **cfg)
              ~~~^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/kaggle/working/mantisv2_kit.py", line 333, in fit
    va = np.mean(predict(va_idx) == y[va_idx])
                 ~~~~~~~^^^^^^^^
  File "/kaggle/working/mantisv2_kit.py", line 317, in predict
    preds.append(model(batch(idx[i:i + 256])).argmax(1).cpu().numpy())
                 ~~~~~^^^^^^^^^^^^^^^^^^^^^^
  File "/usr/local/lib/python3.13/dist-packages/torch/nn/modules/modul
e.py", line 1779, in _wrapped_call_impl
    return self._call_impl(*args, **kwargs)
           ~~~~~~~~~~~~~~~~^^^^^^^^^^^^^^^^^^
  File "/usr/local/lib/python3.13/dist-packages/torch/nn/modules/modul
e.py", line 1790, in _call_impl
    return forward_call(*args, **kwargs)
  File "/kaggle/working/mantisv2_kit.py", line 300, in forward
    return self.head(self.enc(X.reshape(B * C, 1, L).float()).reshape
(B, C, 512))
                     ~~~~~~~~^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/usr/local/lib/python3.13/dist-packages/torch/nn/modules/modul
e.py", line 1779, in _wrapped_call_impl
    return self._call_impl(*args, **kwargs)
           ~~~~~~~~~~~~~~~~^^^^^^^^^^^^^^^^^^
  File "/usr/local/lib/python3.13/dist-packages/torch/nn/modules/modul
e.py", line 1790, in _call_impl
    return forward_call(*args, **kwargs)
  File "/usr/local/lib/python3.13/dist-packages/mantis/architecture/ve
rsion2.py", line 251, in forward
    x_embeddings = self.tokgen_unit(x)
  File "/usr/local/lib/python3.13/dist-packages/torch/nn/modules/modul
e.py", line 1779, in _wrapped_call_impl
    return self._call_impl(*args, **kwargs)
           ~~~~~~~~~~~~~~~~^^^^^^^^^^^^^^^^^^
  File "/usr/local/lib/python3.13/dist-packages/torch/nn/modules/modul
e.py", line 1790, in _call_impl
```

```
    return forward_call(*args, **kwargs)
  File "/usr/local/lib/python3.13/dist-packages/mantis/architecture/version2.py", line 88, in forward
    embedding = self.layer_norms[i](embedding)
  File "/usr/local/lib/python3.13/dist-packages/torch/nn/modules/module.py", line 1779, in _wrapped_call_impl
    return self._call_impl(*args, **kwargs)
           ~~~~~~~~~~~~~~~~~~^^^^^^^^^^^^^^^^^^
  File "/usr/local/lib/python3.13/dist-packages/torch/nn/modules/module.py", line 1790, in _call_impl
    return forward_call(*args, **kwargs)
  File "/usr/local/lib/python3.13/dist-packages/torch/nn/modules/normalization.py", line 229, in forward
    return F.layer_norm(
           ~~~~~~~~~~~~^
        input, self.normalized_shape, self.weight, self.bias, self.eps
        ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
    )
    ^
  File "/usr/local/lib/python3.13/dist-packages/torch/nn/functional.py", line 2935, in layer_norm
    return torch.layer_norm(
           ~~~~~~~~~~~~~~~~^
        input, normalized_shape, weight, bias, eps, torch.backends.cudnn.enabled
        ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
    )
    ^
torch.OutOfMemoryError: CUDA out of memory. Tried to allocate 6.25 GiB. GPU 0 has a total capacity of 14.56 GiB of which 3.50 GiB is free. Including non-PyTorch memory, this process has 11.06 GiB memory in use. Of the allocated memory 6.46 GiB is allocated by PyTorch, and 4.46 GiB is reserved by PyTorch but unallocated. If reserved but unallocated memory is large try setting PYTORCH_CUDA_ALLOC_CONF=expandable_segments:True to avoid fragmentation.  See documentation for Memory Management  (https://docs.pytorch.org/docs/stable/notes/cuda.html#optimizing-memory-usage-with-pytorch-cuda-alloc-conf)
```

```
23:07:51 HAR.h5 domains: ['gear', 'lgwatch', 'nexus4', 's3', 's3mini']
23:07:51   gear: 370 samples, 6 channels
23:07:51   lgwatch: 2154 samples, 6 channels
23:07:52   nexus4: 4577 samples, 6 channels
23:07:52   s3: 4166 samples, 6 channels
23:07:52   s3mini: 2407 samples, 6 channels
23:07:52 HHAR-ID: train 8717, test 3418, channels 6, original length 5
12, classes 6
23:08:04 features (12135, 6, 512) in 12s
23:09:42 HHAR-ID          rf           0.9834 +/- 0.0005    98s
23:09:48 HHAR-ID          logreg       0.9854 +/- 0.0000    6s
23:10:36 HHAR-ID          head         0.9859 +/- 0.0019  {'epochs':
'[52, 72, 80]'}  47s
23:12:01 HHAR-ID          mixer        0.9886 +/- 0.0010  {'epochs':
'[65, 62, 87]', 'gate': '[0.173, -0.165, 0.183]'}  85s
23:12:18 tokens (12135, 6, 6, 5, 256) (1.1 GB) in 17s
23:13:34 HHAR-ID          pool         0.9871 +/- 0.0010  {'epochs':
'[34, 51, 48]', 'gate': '[0.0, 0.0, 0.0]', 'layer_weights': '[[0.161,
0.164, 0.165, 0.164, 0.176, 0.17], [0.164, 0.174, 0.188, 0.154, 0.174,
0.146], [0.164, 0.174, 0.181, 0.158, 0.173, 0.15]]'}  77s
23:15:54 HHAR-ID          pool_mixer   0.9885 +/- 0.0005  {'epochs':
'[75, 55, 59]', 'gate': '[0.283, -0.25, 0.245]', 'layer_weights':
'[[0.175, 0.19, 0.195, 0.148, 0.161, 0.131], [0.17, 0.178, 0.185, 0.15
4, 0.168, 0.145], [0.169, 0.187, 0.192, 0.152, 0.164, 0.137]]'}  139s
23:30:41 HHAR-ID          lora         0.9902 +/- 0.0007  {'epochs':
'[25, 30, 30]', 'gate': '[-0.144, -0.162, -0.155]'}  888s
23:30:41 HHAR-ID done in 22.8 min, saved /kaggle/working/results/HHAR-
ID_results.csv
dataset     method   mean    std                 per_seed
HHAR-ID         rf 0.9834 0.0005 [0.9839, 0.9827, 0.9836]
HHAR-ID     logreg 0.9854 0.0000                 [0.9854]
HHAR-ID       head 0.9859 0.0019 [0.9833, 0.9877, 0.9865]
HHAR-ID      mixer 0.9886 0.0010  [0.988, 0.9877, 0.9901]
HHAR-ID       pool 0.9871 0.0010  [0.9871, 0.9883, 0.986]
HHAR-ID pool_mixer 0.9885 0.0005  [0.9892, 0.9883, 0.988]
HHAR-ID       lora 0.9902 0.0007 [0.9895, 0.9901, 0.9912]
23:30:41 HHAR-OOD: device cuda (Tesla T4)
23:30:41 HAR.h5 domains: ['gear', 'lgwatch', 'nexus4', 's3', 's3mini']
23:30:41   lgwatch: 2154 samples, 6 channels
23:30:41   nexus4: 4577 samples, 6 channels
```

```
23:30:42   s3: 4166 samples, 6 channels
23:30:42   s3mini: 2407 samples, 6 channels
23:30:42 HHAR-OOD: train 11150, test 2154, channels 6, original length
512, classes 6
23:30:55 features (13304, 6, 512) in 13s
23:32:54 HHAR-OOD         rf           0.5868 +/- 0.0038    119s
23:32:58 HHAR-OOD         logreg       0.5604 +/- 0.0000    4s
23:33:09 HHAR-OOD         head         0.5789 +/- 0.0118  {'epochs':
'[40, 40, 40]'}  11s
23:33:24 HHAR-OOD         mixer        0.5195 +/- 0.0135  {'epochs':
'[40, 40, 21]', 'gate': '[0.145, 0.149, 0.12]'}  15s
23:33:43 tokens (13304, 6, 6, 5, 256) (1.2 GB) in 19s
23:34:09 HHAR-OOD         pool         0.5319 +/- 0.0121  {'epochs':
'[13, 20, 20]', 'gate': '[0.0, 0.0, 0.0]', 'layer_weights': '[[0.165,
0.164, 0.163, 0.166, 0.171, 0.171], [0.164, 0.165, 0.164, 0.164, 0.17
1, 0.172], [0.165, 0.165, 0.165, 0.16, 0.172, 0.173]]'}  26s
23:34:41 HHAR-OOD         pool_mixer   0.5402 +/- 0.0255  {'epochs':
'[20, 20, 20]', 'gate': '[0.158, 0.152, 0.153]', 'layer_weights':
'[[0.166, 0.167, 0.165, 0.165, 0.168, 0.169], [0.166, 0.167, 0.165, 0.
166, 0.168, 0.168], [0.166, 0.167, 0.167, 0.166, 0.167, 0.167]]'}  32s
23:37:16 HHAR-OOD         lora         0.5704 +/- 0.0164  {'epochs':
'[4, 4, 4]', 'gate': '[0.073, -0.102, -0.113]'}  155s
23:37:16 HHAR-OOD done in 6.6 min, saved /kaggle/working/results/HHAR-
OOD_results.csv
 dataset     method   mean    std                 per_seed
HHAR-OOD         rf 0.5868 0.0038 [0.5915, 0.5868, 0.5822]
HHAR-OOD     logreg 0.5604 0.0000                 [0.5604]
HHAR-OOD       head 0.5789 0.0118  [0.571, 0.5701, 0.5956]
HHAR-OOD      mixer 0.5195 0.0135 [0.5009, 0.5251, 0.5325]
HHAR-OOD       pool 0.5319 0.0121 [0.5167, 0.5464, 0.5325]
HHAR-OOD pool_mixer 0.5402 0.0255 [0.5436, 0.5074, 0.5696]
HHAR-OOD       lora 0.5704 0.0164 [0.5915, 0.5682, 0.5515]
method                head  logreg    lora   mixer    pool  pool_mixer
rf
dataset
EMOPain             0.8469  0.8310     NaN  0.8535  0.8423      0.8394
0.8826
HHAR-ID             0.9859  0.9854  0.9902  0.9886  0.9871      0.9885
0.9834
HHAR-OOD            0.5789  0.5604  0.5704  0.5195  0.5319      0.5402
0.5868
```

MP50	0.7008	0.6689	NaN	0.7126	0.7048	0.6930	0.7092
MP8	0.6852	0.6521	0.6947	0.7109	0.6359	0.6291	0.6756
SelfRegulationSCP1	0.8305	0.8669	0.8544	0.8316	0.7998	0.7952	0.8134
SelfRegulationSCP2	0.5019	0.5111	0.5148	0.4944	0.4963	0.5352	0.5148